



Savitribai Phule Pune University
Centre For Distance and Online Education

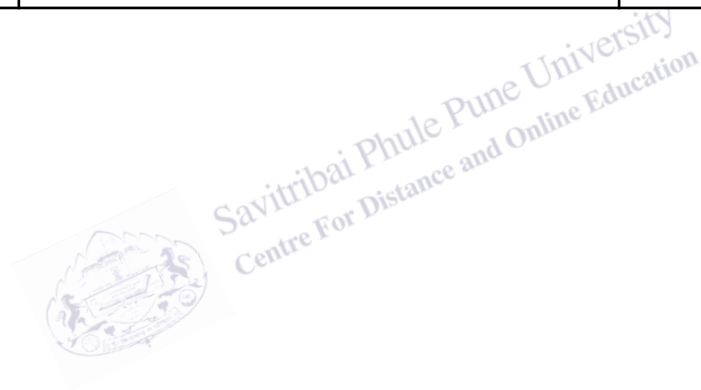
SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	Master of Computer Application	
Course	Programming from first principles	
Topic Name	a structural counterpart to recursive data type definition (practice session) part 2	
Topic Code	-	
Unit/Module No. & Name	12	Recursion
Self-Learning Hours	-	

Topic Details

S.No	Topic	Pg.No
1.0	Introduction	4
2.0	Meaning And Definition	5-7
3.0	Nature Or Type	8-10
4.0	Examples And Case Studies	11-13
5.0	Keywords And Touch Vocabulary	14



OBJECTIVE

1. The main objective of this study material is to help Online MCA students clearly understand the concept of **binary tree traversal using recursion** simply and academically.
2. Trees are an important data structure in computer science, and traversal techniques are essential for accessing and processing tree data effectively.
3. This unit aims to explain how recursion naturally fits the structure of a binary tree, where each node can be treated as a smaller tree.
4. By studying this topic, students will understand how recursive functions visit nodes systematically using in-order, pre-order, and post-order traversal methods.
5. Another objective is to develop logical thinking and problem-solving skills by showing how recursive calls move through left and right subtrees until a base condition is reached.
6. The material also focuses on helping students relate theory to practice through Python code examples that clearly demonstrate traversal logic.
7. This study material further aims to prepare students for examinations by expanding concepts commonly asked in MCQs, such as time complexity, traversal order, and real-world applications.
8. Practical understanding is strengthened through a case study on file system directory traversal, showing how traversal techniques are applied in real computing environments.
9. Overall, the objective is to provide a strong conceptual foundation, improve coding confidence, and enable students to apply binary tree traversal techniques effectively in academic projects and real-life programming tasks.

1.0 INTRODUCTION

Data structures are an important part of computer science because they help organize and store data in an efficient way. Among different data structures, **trees** are widely used because they represent data in a hierarchical form. A **binary tree** is a special type of tree in which each node can have at most two children, called the left child and the right child. Binary trees are commonly used in searching, sorting, file systems, and decision-making applications.

To use the data stored in a binary tree, it is necessary to visit each node in a proper order. This process is known as **tree traversal**. Traversal means accessing and processing every node of the tree exactly once. Since a tree has a recursive structure, traversal techniques are naturally implemented using **recursion**. Recursion is a programming technique in which a function calls itself to solve a smaller part of the same problem.

In binary tree traversal, recursion works by treating each subtree as a smaller tree. The recursive function continues to call itself until it reaches a base condition, usually when the node is **None**. This makes recursion simple, clear, and easy to understand for tree-based problems. Because of this close relationship, recursion is considered the most suitable method for tree traversal.

There are three main types of binary tree traversal: **inorder**, **preorder**, and **postorder**. Each traversal follows a different order of visiting the root node and its children. These traversal methods are important because they are used in many real-world applications such as expression evaluation, copying data structures, deleting files, and searching data efficiently.

Understanding binary tree traversal using recursion helps MCA students build strong logical thinking and problem-solving skills. It also forms a foundation for advanced topics like binary search trees, heaps, graphs, and file system management. Therefore, this topic is an essential part of data structures and programming education.

2.0 MEANING AND DEFINITION

Binary tree traversal refers to the process of visiting and processing each node of a binary tree in a systematic manner. In a binary tree, each node contains data and links to at most two child nodes, known as the left child and the right child. Traversal is necessary because data in a tree is not stored in a linear sequence, so a proper method is required to access all nodes.

Recursion plays a key role in binary tree traversal because a tree has a self-similar structure. Each subtree of a binary tree is itself a smaller binary tree. Due to this property, recursive functions are well suited for tree traversal. A recursive traversal function processes the current node and then calls itself on the left and right subtrees until a stopping condition is reached.

Definition:

Binary tree traversal using recursion can be defined as a technique in which a recursive function is used to visit every node of a binary tree exactly once by repeatedly applying the same logic to its subtrees until a base condition, usually a **None** node, is encountered.

There are three commonly used recursive traversal methods: inorder, preorder, and postorder. Each method follows a specific sequence for visiting the root node and its children. The choice of traversal depends on the requirement of the application, such as sorting data, copying a tree, or deleting nodes safely.

Understanding the meaning and definition of binary tree traversal helps students clearly identify how recursion simplifies complex tree operations. It also helps in solving examination questions, writing correct programs, and applying traversal techniques in real-world computing problems such as file system navigation and expression evaluation.

2.1 Meaning of Binary Tree Traversal

Binary tree traversal means visiting every node of a binary tree exactly once in a specific order. In a binary tree, each node may have a left child, a right child, or no children at all. Traversal ensures that no node is skipped and that data stored in the tree can be read, processed, or modified.

Traversal is essential for operations such as searching values, printing tree elements, copying tree structures, and deleting nodes. Without traversal techniques, it would be difficult to work with tree-based data efficiently.

2.1.1 Role of Recursion in Tree Traversal

Recursion is a natural technique for traversing binary trees because a tree follows a self-similar structure. Each subtree of a binary tree is itself a smaller binary tree. A recursive function solves the problem by calling itself on these smaller subtrees.

In recursive traversal, the function first checks a base condition, usually when the node is **None**. If the node exists, the function processes the node and then makes recursive calls on the left and right children. This process continues until all nodes are visited.

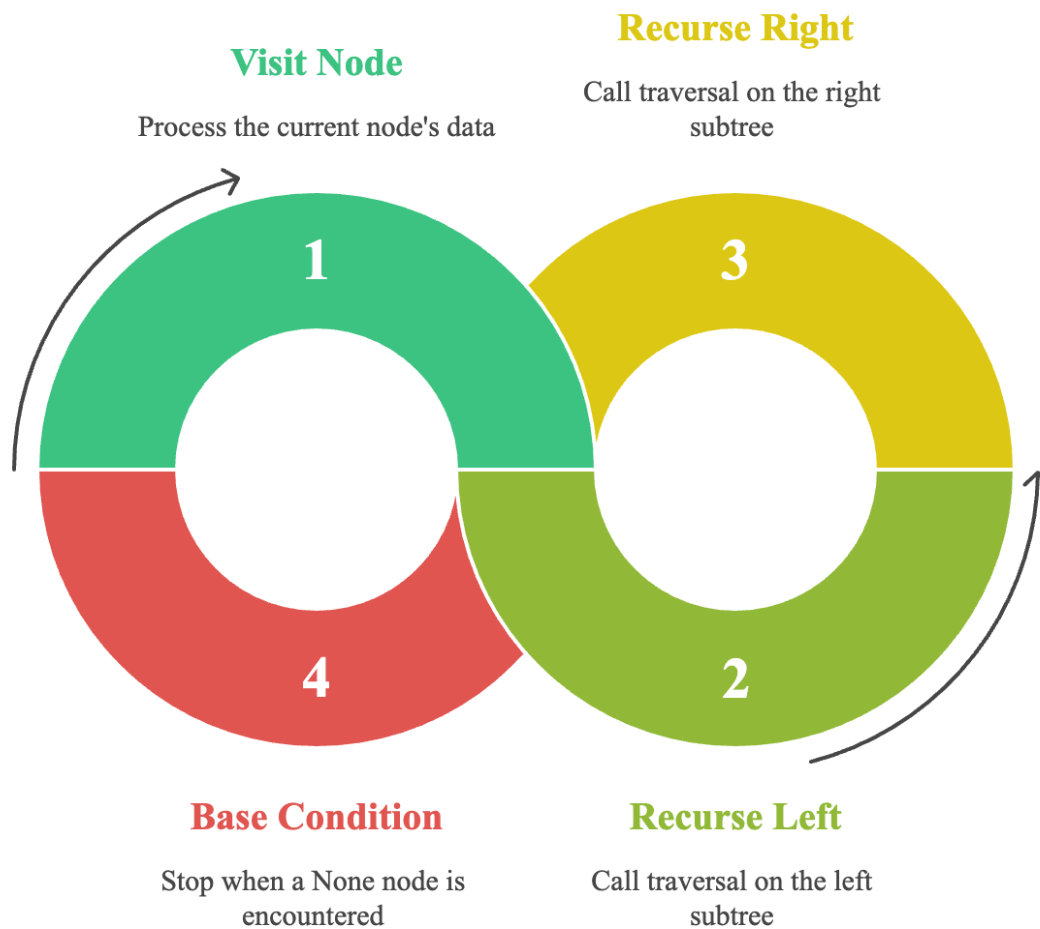
2.2 Definition of Binary Tree Traversal Using Recursion

Binary tree traversal using recursion can be defined as a method in which a recursive function visits all nodes of a binary tree by repeatedly applying the same logic to each subtree until a termination condition is reached.

This definition highlights two key ideas: recursion and systematic visiting of nodes. The recursive approach makes traversal code shorter, clearer, and easier to understand, especially for hierarchical data structures like trees.

Fig. 1

Binary Tree Traversal Cycle



*The diagram shows the recursive traversal cycle in a binary tree: first process the current node, then recursively traverse the left subtree, followed by the right subtree, and finally stop when a **None** node is reached as the base condition.*

2.2.1 Types of Recursive Tree Traversal

There are three main types of recursive binary tree traversal methods: inorder traversal, preorder traversal, and postorder traversal. Each method differs in the order in which the root node and its child nodes are processed.

These traversal types are used based on application needs, such as sorting data, evaluating expressions, copying tree structures, or deleting nodes safely. Understanding these types helps students apply the correct traversal technique in practical and examination scenarios.

3.0 NATURE OR TYPE

The nature of binary tree traversal using recursion explains how traversal works and why recursion is suitable for tree structures. Binary tree traversal is systematic, logical, and follows a fixed order. Since a binary tree is made up of smaller subtrees, recursion helps in breaking the problem into smaller and simpler parts.

Traversal using recursion is not a one-time process. It continues until all nodes of the tree are visited. This makes traversal a complete and reliable method for accessing tree data. Recursive traversal is widely used because it closely matches the structure of a tree.

3.1 Recursive Nature of Binary Tree Traversal

Binary tree traversal is recursive in nature because each node can be treated as the root of a smaller tree. When a recursive function is called on a node, it performs the same steps on its left and right subtrees.

The recursive process continues until it reaches a base condition, which usually occurs when the node is **None**. This base condition prevents infinite recursion and ensures that the function stops correctly. Because of this structure, recursion provides a clean and readable solution for tree traversal problems.

3.1.1 Self-Similar Structure of Trees

A binary tree has a self-similar structure, meaning that every subtree looks like a smaller version of the original tree. Each subtree contains a root node and may have its own left and right children.

This self-similar nature allows the same traversal logic to be applied repeatedly. The recursive function does not need different logic for different parts of the tree. This makes recursion an efficient and natural choice for tree traversal.

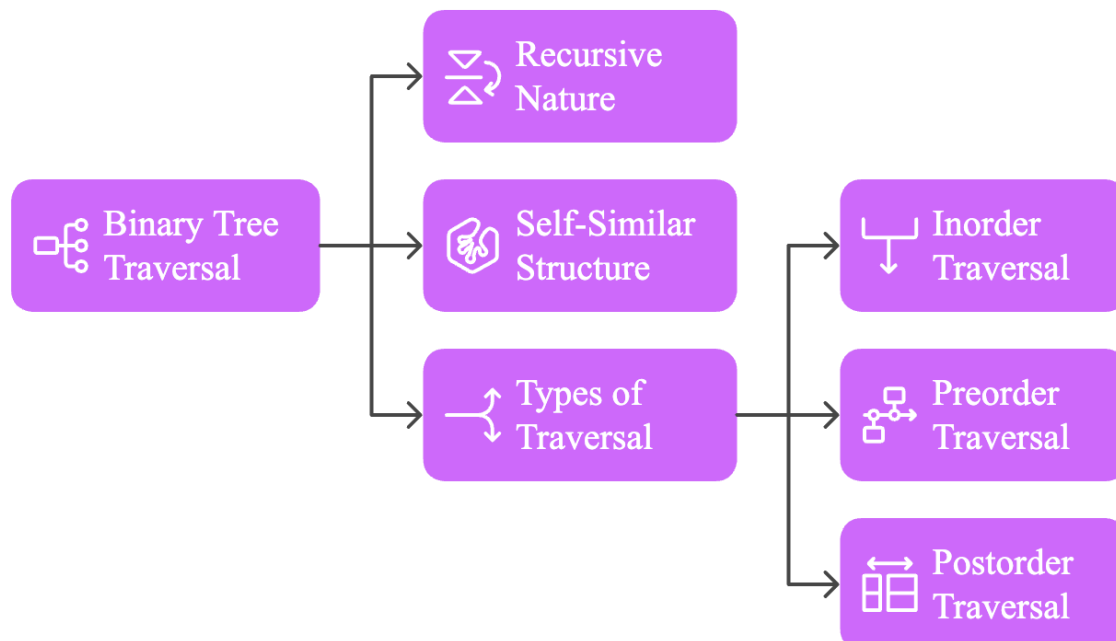
3.2 Types of Binary Tree Traversal

Binary tree traversal can be classified into three main types based on the order in which nodes are visited. These types are inorder traversal, preorder traversal, and postorder traversal. Each traversal type follows a different sequence but visits every node exactly once.

The type of traversal chosen depends on the requirement of the task. For example, some applications require data to be processed before visiting children, while others require children to be processed first.

Fig. 2

Binary Tree Traversal Types



The diagram explains that binary tree traversal follows a recursive and self-similar structure. Based on this nature, traversal is classified into three main types: inorder, preorder, and postorder traversal, each differing in the order of visiting the root and child nodes.

3.2.1 Inorder Traversal

In inorder traversal, the left subtree is visited first, followed by the root node, and then the right subtree. This traversal method is commonly used in binary search trees because it displays data in sorted order.

Inorder traversal is useful when the objective is to retrieve data in a natural or ascending sequence. It is widely used in searching and sorting operations.

3.2.2 Preorder Traversal

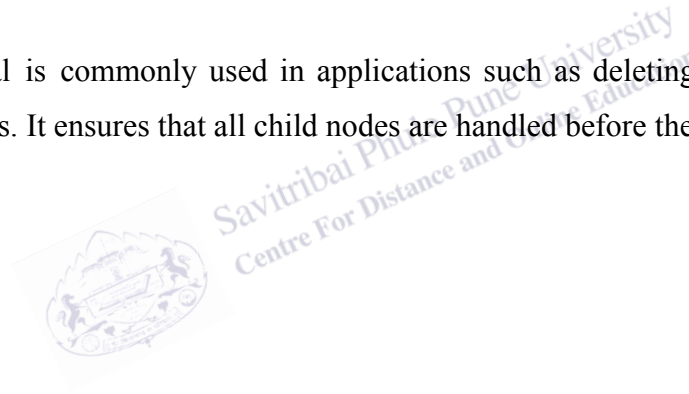
In preorder traversal, the root node is visited first, followed by the left subtree and then the right subtree. This traversal method is useful for creating a copy of the tree or representing expressions in prefix form.

Preorder traversal ensures that the root of the tree is processed before its children, making it suitable for tasks that require early processing of nodes.

3.2.3 Postorder Traversal

In postorder traversal, the left subtree is visited first, then the right subtree, and finally the root node. This traversal method is useful when child nodes must be processed before the parent node.

Postorder traversal is commonly used in applications such as deleting a tree or evaluating postfix expressions. It ensures that all child nodes are handled before the parent node.



4.0 EXAMPLES AND CASE STUDIES

Examples and case studies help students understand how binary tree traversal using recursion is applied in practical situations. While theoretical knowledge explains how traversal works, real examples show why it is important and how it is used in real computing systems. Through examples, students can clearly see the flow of recursive calls and the order in which nodes are processed.

4.1 Example of Binary Tree Traversal

Consider a simple binary tree where each node stores an integer value. The root node contains the value 10. Its left child is 25, and its right child is 15. The left child of 25 is 20, and the right child is 35. The right child of 15 is 30, and the right child of 30 is 45.

When **inorder traversal** is applied to this tree, the function first visits the leftmost node, which is 20. It then moves upward to process 25, followed by 35. After completing the left subtree, the root node 10 is processed. The traversal then continues with the right subtree, visiting 15, 30, and finally 45. This produces the output sequence in a logical and structured order.

In **preorder traversal**, the root node 10 is visited first. The traversal then moves to the left subtree and processes 25 and 20, followed by 35. After completing the left side, the right subtree nodes 15, 30, and 45 are visited. This traversal is useful when the structure of the tree must be preserved.

In **postorder traversal**, the traversal begins with the left subtree. Nodes 20, 35, and 25 are processed first. The right subtree nodes 45, 30, and 15 are then visited. Finally, the root node 10 is processed. This traversal ensures that child nodes are handled before their parent.

4.2 Case Study: File System Directory Tree

A real-life example of binary tree traversal using recursion can be seen in a file system directory structure. In a computer system, folders and files are arranged in a hierarchical manner similar to a tree. Each folder can contain subfolders and files, which makes the structure suitable for tree representation.

When a program needs to copy a directory, it often uses **preorder traversal**. The root folder is created first, and then recursive calls are made to copy all subfolders and files inside it. This ensures that the directory structure is maintained correctly.

When deleting a directory, **postorder traversal** is commonly used. Files and subfolders are deleted first, and the parent folder is deleted at the end. This approach prevents errors such as “directory not empty,” which commonly occur if a folder is deleted before its contents.

4.3 Case Study: Expression Evaluation

Binary tree traversal is also used in expression evaluation. Mathematical expressions can be represented as binary trees where operators are internal nodes and operands are leaf nodes.

In this case, **postorder traversal** is used to evaluate expressions. The operands are processed first, followed by the operator. This approach matches postfix expression evaluation and ensures correct calculation of results.

4.4 Case Study: Binary Tree Traversal in Search Engine Indexing

Search engines use tree-based data structures to organize and retrieve large amounts of data efficiently. One common application of binary tree traversal using recursion is in **indexing and searching keywords**.

In a simplified model, keywords are stored in a binary search tree where each node represents a word. The left subtree contains words that come before the root word alphabetically, and the right subtree contains words that come after it. This structure allows fast searching and ordered retrieval of data.

When the search engine needs to display keywords in alphabetical order, it uses **inorder traversal**. The recursive traversal visits the left subtree first, then the root keyword, and finally the right subtree. This produces a sorted list of keywords, which helps in efficient indexing and quick search results.

During index creation or backup, **preorder traversal** is used to copy the structure of the keyword tree. The root keyword is processed first, followed by recursive processing of left and right subtrees. This ensures that the entire index structure is preserved correctly.

In cases where outdated keywords must be removed, **postorder traversal** is applied. The recursive process deletes child nodes first and then removes the parent node. This prevents errors and ensures safe deletion of indexed data.

This case study shows how binary tree traversal using recursion plays an important role in real-world systems such as search engines. It highlights the practical value of traversal techniques beyond academic examples and helps MCA students understand their importance in large-scale applications.

1. A binary tree is a hierarchical data structure where each node has at most two children.
2. Tree traversal is the process of visiting every node of a tree exactly once.
3. Recursion is best suited for tree traversal because a tree has a self-similar structure.
4. Inorder traversal visits the left subtree, root, and then the right subtree.
5. Preorder traversal visits the root before its child nodes.
6. Postorder traversal visits child nodes before the root node.
7. Each traversal method has different applications based on processing needs.
8. Recursive traversal continues until a base condition, usually a **None** node, is reached.
9. Time complexity of binary tree traversal is $O(n)$, as each node is visited once.
10. File systems use postorder traversal to delete directories safely.
11. Preorder traversal is useful for copying tree structures.
12. Postorder traversal is commonly used for expression evaluation and deletion operations.
13. Understanding traversal improves problem-solving and coding skills for MCA students.

5.0 KEYWORDS AND TOUCH VOCABULARY

Keyword	Meaning
Binary Tree	A tree data structure with at most two children per node
Node	A basic unit of a tree containing data and links
Root	The topmost node of a tree
Leaf Node	A node with no children
Subtree	A smaller tree formed from any node
Traversal	Process of visiting all nodes in a tree
Recursion	A technique where a function calls itself
Inorder Traversal	Left, Root, Right traversal method
Preorder Traversal	Root, Left, Right traversal method
Postorder Traversal	Left, Right, Root traversal method
Base Condition	Condition that stops recursion
Time Complexity	Measure of algorithm efficiency
File System Tree	Hierarchical structure of folders and files